# Jones Automation Exercise Part B - Ronen Melihov

## a. Problems found

1. **No CVV field:** There is no field for the card security code (the 3 digits on the back).
2. **Card number is captured in the form's own input:** The credit card fields are embedded directly within the application's local DOM instead of a secure payment provider iframe, which violates the most fundamental PCI compliance standards.
3. **The payment amount is shown on the page (30.00):** Verify that the charged amount is set and validated server-side and not trusted from the client, otherwise it could be tampered with before submission.
4. **Input validation should be tested:** In functional testing, verify the form validates its inputs: a valid card number, an expiry date that has not passed, and required fields filled in.
5. **The form is built for the US only:** There is no Country field, the "State or Province" list is US-only, and the postal code ("no dashes") assumes a US format. A global company needs support for different countries: address format, postal code, currency, and tax.
6. **No currency shown next to the amount:** "30.00" is ambiguous: dollars? euros? The currency should be displayed.
7. **Formatting burden on the user:** "No dashes or spaces" / "no dashes" force the user to format the input themselves, instead of the system accepting and normalizing it.
8. **Required fields are marked only by a red asterisk:** Colour alone fails color-blind and screen-reader users; add a clear "(required)" label and a programmatic required attribute.
9. **The "Continue" button is not explicit enough:** At the final payment step, the button should reflect the actual action, for example "Pay $30.00" or "Complete Payment".
10. **The MI (Middle Initial) field is a US-specific concept:** Most non-US users won't recognize it. A single "Name on card" field would serve global users better.
11. **The State field has inconsistent copy:** The label reads "State or Province" but the dropdown inside reads "Select a state". Non-US users looking for a province option may be confused.

## b. Sample test cases

| Test Case | Steps | Expected Result |
|---|---|---|
| **Valid payment (happy path)** | On the Account Information page, fill every field with valid data (including CVV once added), a valid future expiry, and a card number that passes the Luhn check, then click Continue. | The form validates, the amount is charged server-side, and the user proceeds to the confirmation page. |
| **Invalid card/expiry (negative validation)** | Enter an invalid card number and a past expiry date, leave required fields empty, then click Continue. | Submission is blocked, clear field-level error messages appear, and no charge request is sent. |
| **Amount integrity (security)** | Before submitting, tamper with the client-side amount (30.00 to 0.01) and inspect the network requests. | The server rejects the transaction due to a price mismatch, or strictly enforces the database price (30.00), returning an error code to the client. Card data travels only over HTTPS and only as a token, never in logs or the URL. |

## c. Most Severe Bug & Product Solution

The entire credit card form is hand-built directly into the UI instead of using a compliant payment provider integration. While the missing CVV field is the most obvious visual flaw, entirely blocks compliance with modern banking standards, leading to massive transaction declines by issuing banks and extreme vulnerability to fraud, the underlying architecture creates significant business and security liabilities:

- **PCI Scope & Legal Liability:** Raw card numbers would pass through your own servers, exposing the company to heavy PCI compliance requirements, potential data leaks, and significant fines.
- **Amount Tampering:** The Payment Amount field appears driven by the client-side UI, making it potentially vulnerable to manipulation before reaching the server.

> **Product Solution:** Replace the hand-built custom fields with a PCI-compliant, tokenized component from an established payment provider such as Stripe Elements, Adyen, or Braintree.
>
> 1. **Security:** Card number and CVV are captured inside the provider's secure iframe. Raw card data never touches your servers, reducing PCI compliance scope to the simplest level.
> 2. **Automated UI & Validation:** The provider's component includes built-in real-time validation and automatic card-type detection, allowing removal of the manual "Card Type" dropdown.
> 3. **Fixes the Missing CVV:** The secure input fields natively include the CVV/CVC field.
> 4. **Prevents Tampering:** The transaction amount is cryptographically bound to a server-side token/session generated prior to checkout, making client-side tampering impossible.
> 5. **Future-Proofing:** Enables 3-D Secure, localized currency, and dynamic postal code validation based on country.